



Technische Universität Berlin  
Fakultät IV - Elektrotechnik und Informatik  
Institut für Energie- und Automatisierungstechnik  
Fachgebiet Elektrische Energiespeichertechnik

Projektdokumentation / Praktikumsdokumentation  
State Estimation and Diagnosis of Battery Systems  
WiSe 25/26

## GroupWork II: Data-Driven RUL Prediction

vorgelegt von:

[REDACTED] ([REDACTED])

[REDACTED] ([REDACTED])

Removed personal data

[REDACTED] ([REDACTED])

Franz Wendel ([REDACTED])

Verantwortliche Hochschullehrerin:  
Betreuer:  
Tag der Einreichung:

Prof. Dr.-Ing. Julia Kowal  
Jiaqi Yao  
June 26, 2026



# Contents

|                                               |           |
|-----------------------------------------------|-----------|
| <b>1 Task 1</b>                               | <b>1</b>  |
| 1.1 Preprocessing of the dataset . . . . .    | 1         |
| 1.2 Purpose of the sets . . . . .             | 1         |
| <b>2 Task 2</b>                               | <b>2</b>  |
| 2.1 LSTMCell VS. LSTM . . . . .               | 2         |
| 2.2 <code>__init__</code> Function . . . . .  | 2         |
| 2.3 <code>forward</code> Function . . . . .   | 3         |
| 2.4 <code>inference</code> Function . . . . . | 4         |
| <b>3 Task 3</b>                               | <b>5</b>  |
| 3.1 <code>train</code> function . . . . .     | 5         |
| 3.2 <code>validate</code> function . . . . .  | 6         |
| 3.3 Selection of Hyperparameters . . . . .    | 6         |
| 3.4 Fine-tuning Hyperparameters . . . . .     | 7         |
| <b>4 Task 4</b>                               | <b>11</b> |
| 4.1 Single-Test Procedure . . . . .           | 11        |
| 4.2 Visualization of SOH Prediction . . . . . | 11        |
| 4.3 RUL Calculation . . . . .                 | 12        |
| 4.4 Impact of History Length . . . . .        | 12        |
| 4.5 Discussion . . . . .                      | 13        |
| <b>Bibliography</b>                           | <b>14</b> |

# 1 Task 1

## 1.1 Preprocessing of the dataset

The NASA lithium-ion battery dataset (.mat files) is loaded in the root folder. To extract the relevant data, it is filtered by cycle type "discharge". The capacity field is read and paired with a cycle index. The state of health (SOH) is computed as  $SOH = \frac{Capacity}{NominalCapacity} \in [0, 1]$ . The nominal capacity is 2.0 Ah. The .csv with Cycle, Capacity and SOH columns is written for each cell. The SOH vs Cycle relation of each cell is visualized in Figure 1.1.

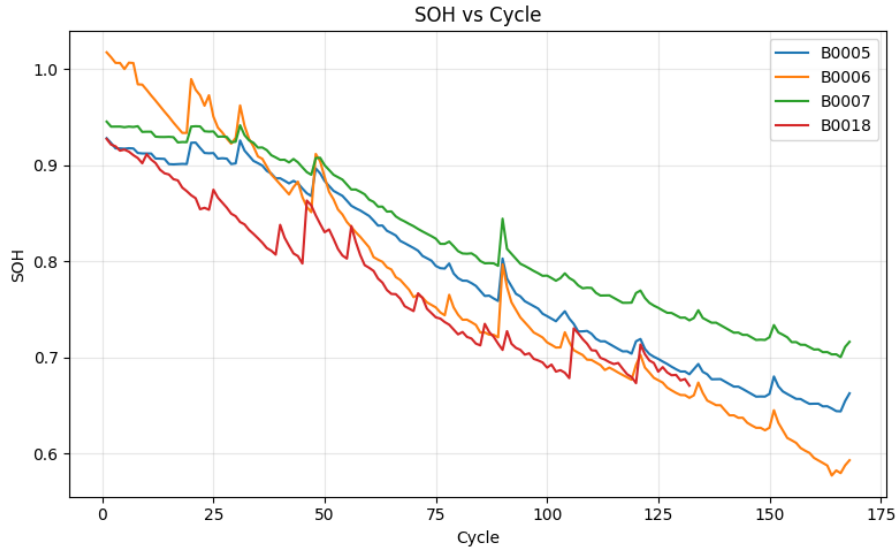


Figure 1.1: SOH vs. Cycles

The data of each cell is added to either the training, validation or test set.

## 1.2 Purpose of the sets

- **Training set:** Used to fit model parameters during training.
- **Validation set:** Used to tune hyperparameters and prevent overfitting.
- **Test set:** Used only for the final evaluation.

## 2 Task 2

### 2.1 LSTMCell VS. LSTM

The difference between `nn.LSTMCell` and `nn.LSTM` is that `nn.LSTMCell` processes one time step at a time and requires manual handling of the hidden and cell states, whereas `nn.LSTM` processes entire sequences automatically and manages the recurrence internally.

Using `LSTMCell` provides fine-grained control over the computation at each time step, which is useful for custom logic, conditional operations, or inspection of intermediate states. In contrast, `nn.LSTM` is more efficient and feature-rich: it supports multi-layer stacking, bidirectionality, dropout between layers, and benefits from optimized backend implementations.

### 2.2 `__init__` Function

To complete the `__init__` function the given `input_size` and the `hidden_size` must be passed into the `nn.LSTMCell` initialization. The `output_size` is used in the definition of the fully connected layer setup.

---

```
def __init__(self, input_size, hidden_size, output_size):
    ...
    # define the LSTM cell
    self.lstm_cell = nn.LSTMCell(input_size=input_size,
                                  hidden_size=hidden_size)
    # define the fully connected layer with Sigmoid activation function
    self.fc = nn.Sequential(
        nn.Linear(hidden_size, output_size),
        nn.Sigmoid()
    )
    ...
```

---

## Sigmoid Activation Function

The sigmoid activation function is well suited for predicting the State of Health (SOH) because it maps any real-valued input to the bounded interval  $[0, 1]$ . Since SOH is defined as a normalized value in the range  $[0, 1]$ , the sigmoid ensures physically meaningful outputs without requiring additional clipping or scaling. Furthermore, its smooth and differentiable nature supports stable gradient-based training, enabling the model to learn faster.

## 2.3 forward Function

---

```
def forward(self, inputs, teacher_forcing_ratio=0.5):
    ...
    # iterate over each time step
    for t in range(seq_len):
        # determine the input for the current time step
        if random.random() > teacher_forcing_ratio and t > 0:
            input_t = output_t
        else:
            input_t = inputs[:, t, :]

        # calculate the hidden state and cell state for the current time
        step
        if t == 0:
            h_t, c_t = self.lstm_cell(input_t, (h_0, c_0))
        else:
            h_t, c_t = self.lstm_cell(input_t, (h_t, c_t))

        # map the LSTM hidden state to the output for the current time
        step using the fully connected layer
        output_t = self.fc(h_t)
        predictions[:, t, :] = output_t # store the output in the
        predictions tensor
    ...
```

---

The prediction tensor is populated with the model outputs at each time step. When teacher forcing is applied, the model input is stochastically chosen to be either the ground-truth value or the model's previous prediction. The probability of selecting each option is controlled by the teacher forcing ratio, which balances guided learning and autoregressive behavior.

## 2.4 inference Function

---

```
def inference(self, history, eol_soh=0.7, max_len=200):
    ...
    for t in range(max_len):
        # determine the input for the current time step
        if t < history_len:
            input_t = history[:, t, :]
        else:
            input_t = output_t

        # calculate the hidden state and cell state for the current time
        step
        if t == 0:
            h_t, c_t = self.lstm_cell(input_t, (h_0, c_0))
        else:
            h_t, c_t = self.lstm_cell(input_t, (h_t, c_t))

        # determine the output for the current time step
        if (t + 1) < history_len:
            output_t = history[:, t + 1, :]
        else:
            output_t = self.fc(h_t)
        predictions.append(output_t) # store the output in the predictions
        list
    ...
```

---

During inference, the model is first provided with the available historical data of the battery to allow it to align with the characteristics of the new cell. Once no further historical measurements are available, the model operates in an autoregressive manner, using its previous prediction as the next input. This phase is critical, as it determines the accuracy and stability of the long-term SOH degradation forecast.

## 3 Task 3

### 3.1 train function

At the beginning of the training step, the input and target sequences are sliced to align the model predictions with the corresponding ground-truth values used for loss computation:

---

```
inputs = SOHs[:, :-1, :]  
targets = SOHs[:, 1:, :]
```

---

Both tensors retain all sequences and feature dimensions. For the input tensor, the final time step is removed because it represents the value to be predicted. Conversely, the target tensor excludes the first time step, since the model is trained to predict the next SOH value given the current one. Thus, each input at time step  $t$  is paired with the target at time step  $t + 1$ .

The training step is then defined as follows:

---

```
outputs = model.forward(inputs)  
loss = loss_fn(outputs, targets)  
loss.backward()  
optimizer.step()  
optimizer.zero_grad()
```

---

First, the sliced input sequence is passed through the model to obtain the predicted outputs. These predictions are compared with the ground-truth target sequence to compute the loss. During backpropagation, the gradients of the loss with respect to all model parameters are calculated. The optimizer then updates the model parameters using these gradients and the specified learning rate. Finally, the gradients are reset to prevent accumulation across training iterations.



## 3.2 validate function

The input and target sequence are sliced as in the training function. After that, the inputs are passed to the model, and the predicted output is compared to the ground truth of the target sequence. The final step of the validation function is to calculate the average loss over all batches.

## 3.3 Selection of Hyperparameters

In this task, preparations for the fine tuning of the model are being made. To tune the model, 3 hyperparameters were chosen.

The learning rate determines how large the parameter updates are during training. If it is too small, the training progresses very slowly and may not reach a good solution within the available epochs. If it is too large, updates can overshoot good solutions, cause oscillations, or even make training diverge, leading to poor performance.

The hidden size sets the dimensionality of the LSTM's hidden state and cell state, which affects the number of learnable parameters. Increasing the hidden size increases model capacity but also raises computation and memory costs. If the hidden size is too small, the LSTM may lack capacity and may not be able to model the patterns of the given task. If the hidden size is too large, the model may overfit by learning training-specific patterns, leading to worse generalization on validation and test data.

The number of epochs determines how many full passes over the training set are performed. This choice depends on factors such as task difficulty, model capacity, the amount of training data, and training settings such as learning rate. Training for too many epochs can lead to overfitting, where validation performance stops improving or worsens while training loss continues to decrease. Techniques like early stopping and saving the best validation checkpoint can help prevent this, but are not used in this simple implementation.

## 3.4 Fine-tuning Hyperparameters

For the fine tuning of the model a grid search was performed over the previously discussed hyperparameters. The model was trained with all combinations of the parameters listed below.

---

```
learning_rate_list = [1e-4, 1e-3, 1e-2]
num_epochs_list = [50, 100, 200, 300]
hidden_size_list = [8, 16, 32, 64, 128, 256]
```

---

### Learning Rate

The smallest learning rate of 0.0001 performed the worst. In particular, when the model was trained for only a small number of epochs, the training and validation loss curves decreased almost linearly (Figure 3.1), indicating that the optimizer could not make sufficient progress toward a minimum within the allotted training time with the chosen learning rate.

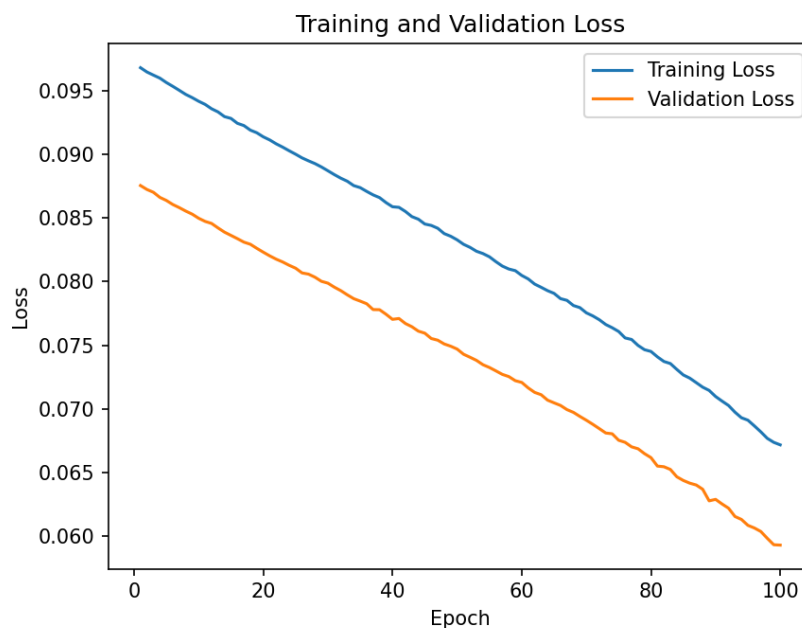


Figure 3.1: Hyperparameters: learning rate: 0.0001, epochs: 100, hidden size: 32

Both learning rates, 0.001 and 0.01, performed better than the smallest learning rate (Figure 3.2). However, each worked best in different settings: when the model had relatively few parameters and training was limited to a small number of epochs, the higher learning rate often achieved better results because it converged more

quickly. In contrast, with sufficient model capacity and training time, the smaller learning rate typically outperformed the others, as it allowed for more stable refinement towards a better minimum.

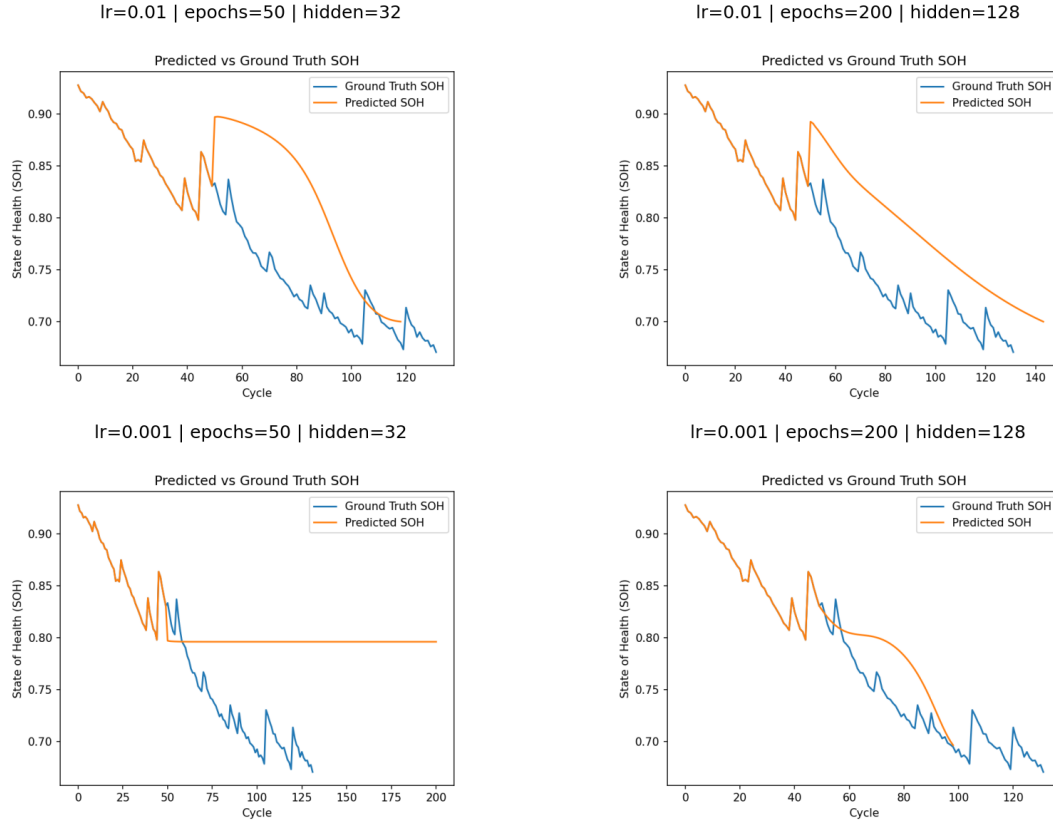


Figure 3.2: Comparison of learning rate 0.01 and 0.001

## Epochs

With the lowest tested value of 50 epochs, the LSTM consistently performed poorly because it did not have enough time to learn suitable parameters from the training data. Increasing the training duration to 100 epochs led to substantial improvements, but beyond that the gains generally became smaller, as can be seen in Figure 3.3. Models with a larger number of parameters still benefited noticeably from additional epochs, whereas smaller models tended to plateau earlier. After about 150 epochs, most configurations showed little to no further improvement.

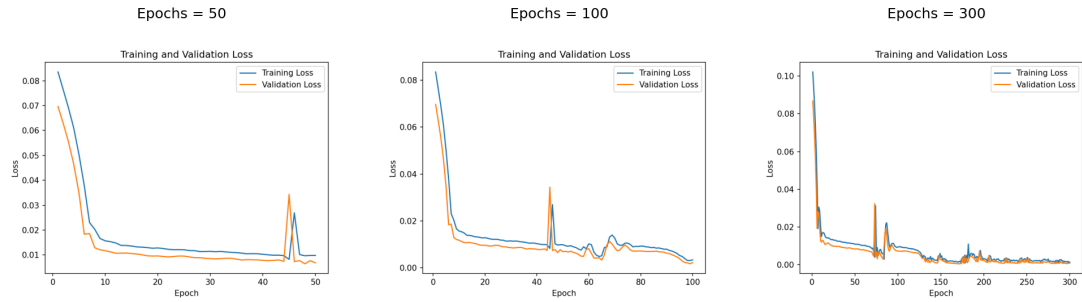


Figure 3.3: Comparison of training and validation loss with learning rate: 0.001 and hidden size: 128

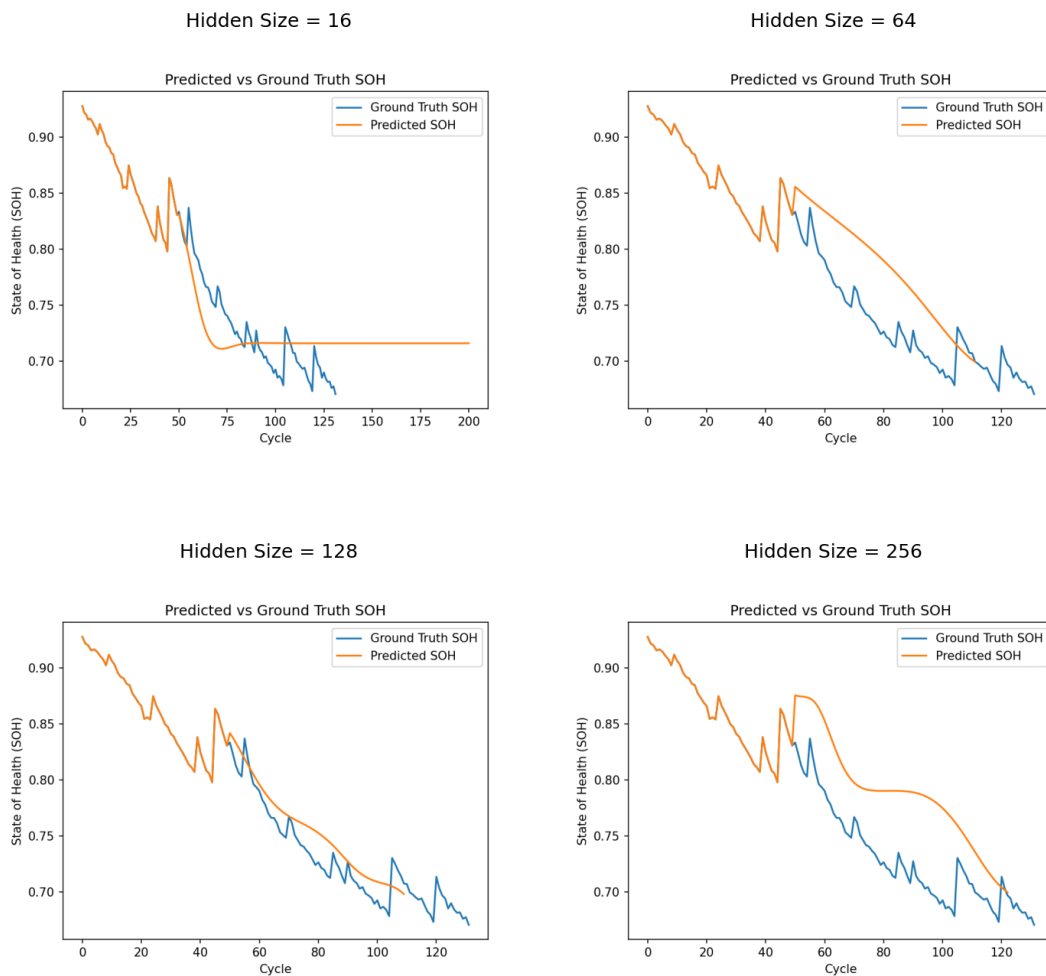


Figure 3.4: Comparison of SOH predictions with parameters learning rate: 0.001, epochs: 200

## Hidden Size

Within the tested range, model performance generally improved as the size of the hidden layer increased, provided that sufficient training time was available. Models

with a small hidden size often lacked the capacity to capture the underlying dynamics and tended to produce linear SOH predictions. In contrast, larger models were better able to represent the SOH trend and produced curves that more closely matched the average shape of the ground-truth trajectory in the validation and training data (Figure 3.4). The model with a hidden size of 256 performed worse, likely because it began to overfit the training data. Additionally, we observed that the inference function produced the same output when using models with a hidden size larger than 256, no matter how much historical data was provided. This is a clear indication that the model overfits and starts to learn the training data based on the cycle count instead of predicting its value based on the given input.

## Teacher Forcing Ratio

In addition to the default teacher forcing ratio (TFR) of 0.5, we evaluated the impact of alternative values (0.2, 0.3, and 0.8) to assess their influence on training dynamics and prediction accuracy. The experiments showed only minor differences in performance across the tested ratios, indicating that the model is relatively insensitive to changes in the teacher forcing probability for this SOH prediction task.

## Resulting parameters

The best-performing model uses:

- Learning rate: 0.001
- Epochs: 200
- Hidden size: 128

## 4 Task 4

In this task, the trained LSTM-based SOH prediction model is evaluated on the test dataset. The objective is to assess the accuracy of recursive SOH prediction and the resulting Remaining Useful Life (RUL) estimation under different amounts of available historical data.

### 4.1 Single-Test Procedure

For evaluation, a single battery degradation profile from the test dataset is selected. The test sequence is first expanded with a batch dimension and then split into a historical segment and a future segment. Given a predefined history length  $L_h$ , the historical SOH data

$$\mathbf{SOH}_{1:L_h}$$

is used as input to the model for recursive inference.

During inference, the model predicts future SOH values step by step, where each predicted SOH value is fed back as input for the next time step. The prediction process continues until either the predefined end-of-life (EOL) SOH threshold is reached or the maximum sequence length is exceeded.

The mean absolute error (MAE) between the predicted SOH sequence  $\hat{\mathbf{SOH}}$  and the ground-truth SOH sequence  $\mathbf{SOH}$  is computed as

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N \left| \mathbf{SOH}_{L_h:i} - \hat{\mathbf{SOH}}_{L_h:i} \right|,$$

where  $N$  denotes the number of time steps from the start of recursive prediction to the end of the valid comparison range.

### 4.2 Visualization of SOH Prediction

The predicted SOH trajectory is plotted together with the ground-truth SOH curve in Figure 4.1. The end of the historical window is indicated by a vertical dashed

line, while the EOL SOH threshold (set to 0.7) is indicated by a horizontal dashed line. This visualization allows for a qualitative comparison between prediction and ground truth and highlights the point where the battery reaches its end of life.

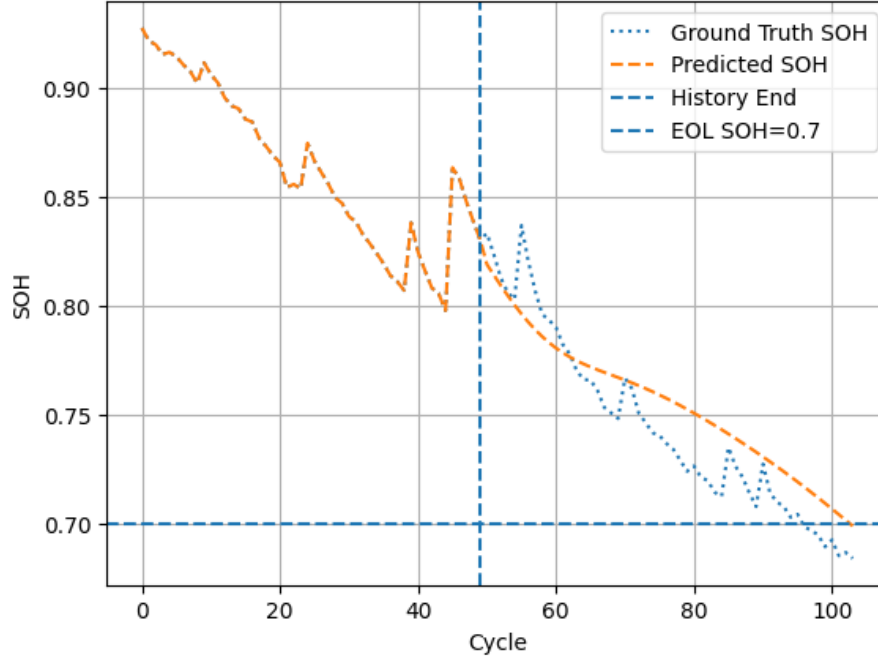


Figure 4.1: SOH prediction compared to ground truth with 50 cycles of historical data

### 4.3 RUL Calculation

The Remaining Useful Life (RUL) is defined as the number of cycles from the end of the historical window until the SOH first falls below the EOL threshold. The ground-truth RUL and predicted RUL are then computed as

$$\text{RUL}_{\text{GT}} = \max(0, i_{\text{EOL,GT}} - L_h), \quad \text{RUL}_{\text{Pred}} = \max(0, i_{\text{EOL,Pred}} - L_h).$$

The RUL prediction error is defined as

$$\text{RUL}_{\text{Error}} = \text{RUL}_{\text{Pred}} - \text{RUL}_{\text{GT}}.$$

### 4.4 Impact of History Length

To analyze the influence of historical data availability on the RUL prediction, the evaluation is repeated with different history lengths. The resulting RUL prediction errors are summarized in Table 4.1.

Table 4.1: RUL prediction error for different history lengths in cycles

| History Length | RUL Error |
|----------------|-----------|
| 10             | 12        |
| 20             | 11        |
| 30             | 10        |
| 40             | 8         |
| 50             | 7         |
| 60             | 8         |
| 70             | 7         |
| 80             | 5         |
| 90             | 3         |

## 4.5 Discussion

The results show a clear trend: increasing the amount of available historical SOH data generally leads to a reduction in RUL prediction error. With short history lengths, the model has limited information about the degradation dynamics, resulting in larger uncertainty in long-term predictions. As the history length increases, the model is better able to capture the degradation trend, leading to more accurate SOH extrapolation and RUL estimation.

However, the improvement is not strictly monotonic, as small fluctuations in error can still be observed. Overall, the lowest RUL error is achieved with the longest history length, demonstrating the importance of sufficient historical data for reliable RUL prediction in data-driven battery health prognostics.



# Bibliography

- [1] Product specification: Rechargeable Lithium Ion Battery 18650HE4 (2013)

# List of Figures

|     |                                                                                                        |    |
|-----|--------------------------------------------------------------------------------------------------------|----|
| 1.1 | SOH vs. Cycles . . . . .                                                                               | 1  |
| 3.1 | Hyperparameters: learning rate: 0.0001, epochs: 100, hidden size: 32                                   | 7  |
| 3.2 | Comparison of learning rate 0.01 and 0.001 . . . . .                                                   | 8  |
| 3.3 | Comparison of training and validation loss with learning rate: 0.001<br>and hidden size: 128 . . . . . | 9  |
| 3.4 | Comparison of SOH predictions with parameters learning rate: 0.001,<br>epochs: 200 . . . . .           | 9  |
| 4.1 | SOH prediction compared to ground truth with 50 cycles of historical<br>data . . . . .                 | 12 |

# List of Tables

|     |                                                                        |    |
|-----|------------------------------------------------------------------------|----|
| 4.1 | RUL prediction error for different history lengths in cycles . . . . . | 13 |
|-----|------------------------------------------------------------------------|----|